

Reducing the Cost of LLM Coding and Reasoning Models with Routers

Aristides Milios and Christopher Pal
ServiceNow AI Research

Abstract

*Routers typically choose a model per query by trading predicted correctness against cost. However, most price each model at a fixed cost, assuming that the number of tokens each model outputs does not vary significantly. We find that the output length of reasoning models varies by an order of magnitude across queries. We predict **both** success probability **and** route-specific output length from the prefill activations of a single small frozen 4B LLM, using a linear readout per route. On 6,500 MMLU-Pro, 1,000 Omni-MATH, and 341 LiveCodeBench tasks, predicted costs reduce spending at matched accuracy by 28.8%, 34.8% and 33.7% relative to median-length pricing, half to three quarters of what perfect per-query cost knowledge would save. They also beat every alternative estimator we evaluated, with paired intervals excluding zero; pricing from difficulty alone matches them only where output length tracks difficulty. We then ablate across different small LLMs and show that every small LLM prefill we tried (0.6–8B, four model families) can be used as a router in this way.*

1. Introduction

Routing reasoning models requires predicting both correctness and cost before generation. Token rates are published; the number of tokens is not. A model that is cheap on average can be expensive on a particular problem, and the outputs of the models we study vary 10–90 $\times$ across problems.

Prefill routers predict correctness from a small encoder’s prompt activations; the prefill router of Varshney et al. [19] explicitly prices outputs at median training length. We read output length from the same activations (Fig. 1). Cost prediction adds one linear readout per route to an existing prefill router, without another encoder pass.

We contribute: (i) evidence on held-out test problems at billed prices that prefill cost readouts reduce spending at matched accuracy and outperform mean-length, prompt-feature, text-embedding and ZeroRouter cost estimators; (ii) a mechanism, from ablations that price from our own success readouts: difficulty suffices where output length

tracks it and fails where it does not; (iii) controls showing the gain is the prefill representation, holds for every small prefill we tried (0.6–8B, four families) and transfers to two further routed model families; and, as a side note, (iv) the same model’s provider endpoints drift, which the readouts can absorb with one offset per endpoint; we pin providers instead.

Related work. Learned routers pick one model per query from the prompt, trading predicted quality against price: RouteLLM learns a strong-versus-weak decision from preference data [14], RouterBench standardizes the evaluation [8], and IRT-Router models query difficulty and model ability with item response theory [18]. Closest to us, the prefill router of Varshney et al. predicts each model’s correctness from a small encoder’s prompt activations [19], and pre-generation activations are known to encode whether a model will fail [12]. These routers, like SWE-Router for agentic software engineering [17], price each model at a per-model constant; the prefill router uses the median training output length, which is our main reference. Fewer routers predict per-query cost: MixLLM regresses output length on query embeddings [20], CARROT uses embedding nearest neighbours or a fine-tuned RoBERTa [16], GraphRouter predicts cost on a query–model graph [4], Route-To-Reason predicts reasoning-model length from text embeddings [15], and ZeroRouter prices queries by item-response difficulty bins [23]. We compare against the embedding-based and ZeroRouter estimators directly, with every estimator given the same success predictions.

Cascades instead call models in sequence and stop when an answer is accepted: FrugalGPT learns a scorer that decides whether to escalate [2], AutoMix uses self-verification [1], and Dekoninck et al. unify routing and cascading [3]. A cascade pays for the cheap attempt on every query; we route once, before generation. Activation-based length prediction also supports serving and scheduling of a single model [13, 22]; we predict the length of other models from one shared prefill. Hosted open-weight models vary across providers in accuracy, length and price [6, 10, 11], and FACET selects among providers of a fixed model per task from measured outcomes [7].

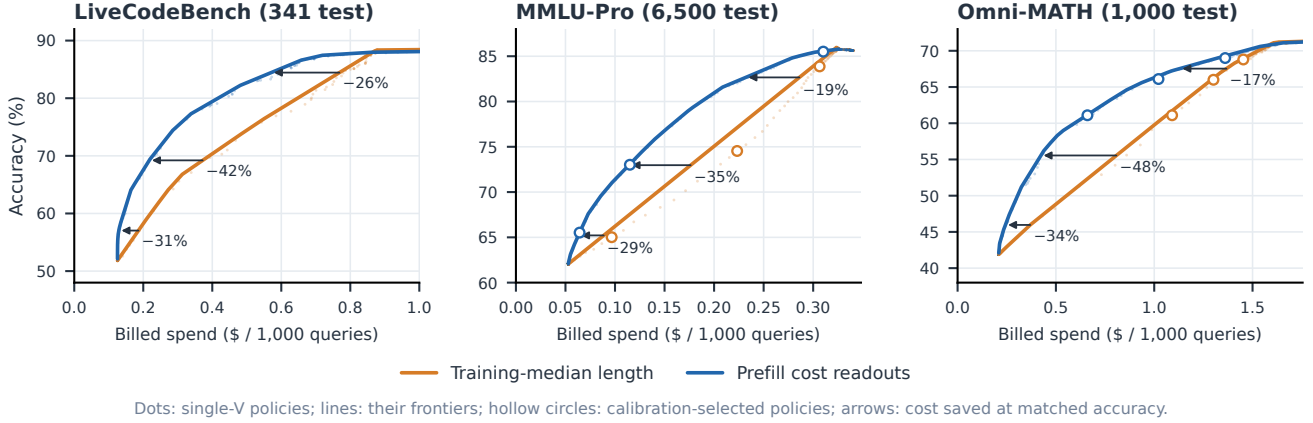


Figure 1. **Cost-accuracy frontiers at billed prices.** Prefill cost readouts (blue) versus training-median length (orange), with the same success predictions and the same encoder pass. Dots: the deterministic policy at each V ; lines: their frontiers; hollow circles: policies selected on calibration for fixed accuracy targets. Arrows run from the median rule’s frontier to ours at matched accuracy and give the cost saved; averaged over the shared accuracy band the saving is 34% on LiveCodeBench, 29% on MMLU-Pro and 35% on Omni-MATH.

2. Cost Prediction and Routing

For query x and route m (a model at a fixed reasoning effort, served by an endpoint), let $p_m(x)$ be the probability of a correct answer and $c_m(x)$ the expected generation cost. One route is chosen before generation,

$$m^*(x; V) = \arg \max_m \{V \hat{p}_m(x) - \hat{c}_m(x)\}, \quad (1)$$

where V is the value of a correct answer; sweeping V trades accuracy for cost. Every query is answered once, with no verifier or fallback.

Readouts. We concatenate mean and last-token activations from eight layers of a frozen Qwen3-4B (Instruct-2507; Thinking-2507 on Omni), standardized on training data. Success readouts are L2-regularized logistic regressions with held-out Platt calibration; cost readouts are ridge regressions of log mean output length, regularized by training-only cross-validation. Predicted length is exponentiated with residual smearing and matched to the route’s training mean, then priced: $\hat{c}_m(x) = r_m^{\text{in}} n_m^{\text{in}}(x) + r_m^{\text{out}} \hat{\ell}_m(x)$.

3. Experimental Protocol

The routes are gpt-oss-20b at low and medium effort, deepseek-v4-flash, and gpt-oss-120b at medium and high effort, called through OpenRouter, with deepseek-v4-flash pinned to one provider (StreamLake) throughout, because its output length depends on the serving provider (see Endpoints). LiveCodeBench [9] has 892 problems split by date into 441 training, 110 calibration and 341 test problems. Omni-MATH [5] and MMLU-Pro [21] have 275/75 and

550/150 training/calibration problems and 1,000 and 6,500 disjoint test problems. Training problems have 2–16 draws per route; Omni-MATH and MMLU-Pro test problems have one. Table 2 gives each route’s accuracy and cost.

Prices. Realized cost is the cost billed for each call (on LCB, realized tokens at the billed rates). Predictions are priced at each model’s effective billed rate, fitted on the test calls (USD per million output tokens: 0.13 for gpt-oss-20b, 0.08 for pinned deepseek-v4-flash, 0.26 for gpt-oss-120b). These differ from the list prices by up to $2.3\times$.

Evaluation. All estimators are fitted on training problems only and evaluated on the test sets, with the same success predictions. Each V on a dense grid gives a deterministic policy and an observed (cost, accuracy) point. Comparing two cost estimators a, b , we interpolate each one’s cost at 12 accuracies spanning the interior 90% of the range both reach and report $1 - \exp(\log C_a/C_b)$, the cost saved by a at matched accuracy. Intervals are percentile intervals from paired problem bootstraps (300 resamples), keeping each problem’s routes together. To check that an operating point can be chosen in advance, we also select the cheapest V reaching each accuracy target on calibration and apply it once.

4. Results

Cost readouts beat every estimator. At matched accuracy, prefill cost readouts spend 33.7% less than median-length pricing on LCB, 34.8% less on Omni-MATH and 28.8% less on MMLU-Pro (Table 1). That is 74%, 63% and 49% of the headroom, the saving perfect per-query cost

Cost saved by ours vs.	LCB ($n=341$)	Omni-MATH ($n=1,000$)	MMLU-Pro ($n=6,500$)
Training-median length	33.7 [24.1, 39.9]	34.8 [27.9, 39.8]	28.8 [26.0, 31.8]
Training-mean length	29.6 [23.6, 34.5]	35.8 [28.9, 40.5]	28.6 [25.9, 31.7]
Prompt-feature GBM	26.3 [16.3, 32.9]	26.5 [18.9, 31.9]	25.2 [21.0, 28.7]
MixLLM-style embeddings	18.2 [11.2, 25.2]	26.7 [20.0, 32.5]	19.2 [15.3, 22.5]
ZeroRouter (reimplemented)	21.1 [13.2, 26.8]	14.4 [8.4, 20.6]	29.0 [25.5, 32.0]
Headroom: oracle vs. median	45.3 [36.6, 50.4]	55.5 [50.4, 58.8]	59.5 [57.8, 61.2]

Table 1. **Cost saved at matched accuracy, billed costs.** Cost saved (%) by our prefill cost readouts relative to each estimator, with 95% paired bootstrap intervals; same success predictions for every arm except ZeroRouter, which uses its own; all intervals exclude zero. Last row: cost saved over median pricing by perfect per-query cost knowledge (each problem priced at its realized output), with the same success predictions; our readouts capture 74%, 63% and 49% of it. Test sets as in Fig. 1. MixLLM-style: jina-embeddings 137M with the MixLLM ensemble head (LCB, code-aware variant). ZeroRouter: item-response latent fitted on the five routes, 4B features, its own success model and bin pricing, configuration chosen on calibration.

Route	LCB		Omni-MATH		MMLU-Pro	
	Acc.	Cost	Acc.	Cost	Acc.	Cost
gpt-oss-20b, low	51.8	0.13	41.9	0.21	62.1	0.05
gpt-oss-20b, medium	68.2	0.65	57.5	1.20	69.4	0.18
deepseek-v4-flash	89.3	0.90	71.3	1.66	86.2	0.33
gpt-oss-120b, medium	82.1	0.64	60.3	1.12	77.2	0.23
gpt-oss-120b, high	89.5	3.73	66.6	4.57	79.3	0.90

Table 2. **Routes on the test sets.** Accuracy (%) and mean billed cost per call (USD per 1,000 calls). No route dominates on every query: costs differ 17–29 $\times$ across routes, and accuracy is not monotone in price.

knowledge would bring (45–60%). Every external estimator loses with intervals excluding zero: mean length by 29–36 points, a prompt-feature model by 25–27, MixLLM-style embeddings by 18–27 and a full ZeroRouter reimplementation by 14–29. Choosing V in advance costs little: on Omni-MATH and MMLU-Pro, policies selected on calibration spend 0–4% more than our own test frontier at the accuracy they reach (hollow circles in Fig. 1).

Endpoints. The same model served by different providers behaves like the model plus an offset: pinning deepseek-v4-flash to each of three providers, they agree on correctness for 90–96% of problems and their output lengths correlate at .83–.95, but one writes about half as many tokens as another. Reusing the readouts with one multiplicative length offset per endpoint, estimated from 50 calls, removes the resulting cost bias on average, though a single estimate is noisy. A few dozen calls per provider also identify the cheapest one, and pinning it beats routing over providers; we therefore pin deepseek-v4-flash throughout, which also makes its length more predictable and raises our saving on

every large set.

Other pools. Savings against median pricing are 14.9% [9.4, 21.4] on AIME 1983–2024 (281 test problems) and 16.8% [5.6, 28.3] on APPS (382).

5. Ablations

Is difficulty enough to price? Two ablations price each route from our own success readouts instead of a dedicated cost readout: ZeroRouter’s bin lookup on mean success logit (ten quantile bins), and a ridge regression of log length on the success logits. Both come within a few points of the cost readout on LCB (ours saves 4.0% [−1.1, 9.0] and −0.8% [−5.6, 4.0]) and on Omni-MATH (6.4% [1.1, 11.6] and −0.1% [−4.6, 4.6]), where output length tracks difficulty, and lose by 22.3% [18.8, 24.7] and 24.1% [21.1, 26.7] on MMLU-Pro. There, even empirical difficulty explains little of log length ($R^2 \approx .19$), while subject labels recover roughly 60% of the gap. The prefill encodes length information beyond difficulty, and that information pays where difficulty and length come apart.

Representation, readout and prefill size. Table 3 crosses three text encoders with three cost readouts: ridge, CARROT-style k -nearest neighbours and a MixLLM-style ensemble, each with our success predictions and with every encoder pass priced. Our readout saves 16–27% against every cell, including Qwen3-Embedding-8B, an encoder twice the prefill’s size. On PCA-reduced 4B features the three readouts are within about 7 points of one another, so the gain comes from the representation rather than the regressor. Pricing the encoder pass changes no comparison by more than 3 points. Table 4 varies the prefill model while keeping prompts, layers and both readouts fixed. Every size saves 22–34% against median pricing, and length R^2 rises with size. None beats our prefill: Qwen3-1.7B and 8B come within 0.3–4 points (within the intervals on Omni-MATH), while 0.6B trails by 9–11. The result is not specific to Qwen: prefills from three other families (Phi-4-mini, Granite-3.3-2B, SmolLM2-1.7B) also save 25–31% against median pricing on both sets. Phi-4-mini matches our router on MMLU-Pro (−0.7 [−2.6, 1.3]) but trails by 10 points on Omni-MATH; Granite and SmolLM2 trail by 5–9.

New model families. Readouts for Qwen3-32B and GLM-4.7-flash, fitted on the same frozen 4B features, were evaluated on 1,905 MMLU-Pro test problems (log-length R^2 .48 and .06). With all seven routes, prefill cost readouts save 30.0% [25.4, 35.3] relative to median pricing, 21.8% [16.3, 26.7] relative to cost from success and 20.6% [15.3,

Encoder	Omni-MATH			MMLU-Pro		
	Ridge	kNN	MixLLM	Ridge	kNN	MixLLM
Qwen3-Embedding-8B	16.6	20.9	21.5	21.1	21.3	19.5
jina-embeddings 137M	23.8	25.3	26.6	17.8	20.4	18.9
MiniLM-L12	24.3	25.7	25.4	18.8	19.0	16.2

Table 3. **Representation and readout.** Cost saved (%) by our prefill readouts relative to each encoder-readout cell at matched accuracy on the Omni-MATH and MMLU-Pro test sets; billed costs, every encoder pass priced, same success predictions. Every paired bootstrap interval (200 resamples) excludes zero.

Prefill	Omni-MATH			MMLU-Pro		
	R^2	vs. median	vs. ours	R^2	vs. median	vs. ours
Qwen3-0.6B	.62	31.9	−9.0	.33	22.0	−10.8
Qwen3-1.7B	.65	34.4	−2.8	.42	26.8	−3.0
Qwen3-4B (hybrid)	.67	29.3	−4.1	.45	25.8	−4.4
Qwen3-8B	.68	31.4	−0.3	.47	26.1	−4.1
Phi-4-mini (3.8B)	.65	31.4	−10.2	.45	28.4	−0.7
Granite-3.3 (2.5B)	.62	26.6	−9.4	.39	25.8	−4.7
SmolLM2 (1.7B)	.59	31.4	−7.6	.33	24.8	−4.9
Qwen3-4B-2507 (ours)	.70	34.8	−	.50	28.8	−

Table 4. **Prefill size.** Each prefill feeds both readouts, refitted with the paper’s recipes on the training problems; Omni-MATH and MMLU-Pro test sets, billed costs. R^2 : test log-length R^2 averaged over routes. vs. median: cost saved relative to median pricing with the same row’s success predictions. vs. ours: cost saved by the row’s router relative to ours (negative: it spends more); its paired intervals (300 resamples) include zero for Qwen3-1.7B and 8B on Omni-MATH and Phi-4-mini on MMLU-Pro.

25.4] relative to difficulty bins. On a pool of the two gpt-oss-20b routes and the two new families, they save 32.2%, 11.6% and 14.8%. Readouts onboarded from ten examples per new route match the full readouts in the seven-route pool (0.1 points) and trail them by 11.9 [−4.2, 21.9] in the new-family pool.

6. Discussion and Limitations

Each route needs output-length labels. The main experiments use five routes from two model families; two further families (Qwen3-32B, GLM-4.7-flash) were tested on MMLU-Pro only. Omni-MATH and MMLU-Pro test problems have one draw per route. Intervals are pointwise and assume independent problems, and Table 1 conditions on the fitted readouts; resampling the training problems as well and refitting every estimator widens them, but our saving over median pricing stays positive on both large test sets (Omni-MATH [23.2, 38.0], MMLU-Pro [22.9, 30.7]). Only deepseek-v4-flash is pinned to a provider; the gpt-oss routes are not, since their provider changes price but barely length. The prefill comparison covers four model families at up to 8B. Billed rates were measured over one collection, and

prices change within days. The ZeroRouter, MixLLM and CARROT baselines are reimplementations.

7. Conclusion

A frozen prefill that already predicts success also predicts cost. On held-out test problems at billed prices, its cost readouts reduce spending at matched accuracy and outperform every alternative estimator we tested; pricing from difficulty alone matches them only where output length tracks difficulty. Endpoints of one model share these readouts up to an offset, so the readouts can absorb provider drift from a few dozen calls, though pinning a provider is simpler.

References

- [1] Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, et al. AutoMix: Automatically Mixing Language Models, 2023. arXiv:2310.12963.
- [2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance, 2023. arXiv:2305.05176.
- [3] Jasper Dekoninck, Maximilian Baader, and Martin Vechev. A Unified Approach to Routing and Cascading for LLMs, 2024. arXiv:2410.10347.
- [4] Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A Graph-based Router for LLM Selections, 2024. arXiv:2410.03834.
- [5] Bofei Gao, Feifan Song, Zhe Yang, et al. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models, 2024. arXiv:2410.07985.
- [6] Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations*, 2025.
- [7] Liang He, Jingbo Wen, Yixiong Chen, Yue Yang, Qizhen Lan, Kangning Cui, and Xilu Wang. You cannot pick a provider from the price list: Market-aware routing for open-weight LLM inference, 2026.
- [8] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, et al. Router-Bench: A Benchmark for Multi-LLM Routing System, 2024. arXiv:2403.12031.
- [9] Naman Jain, King Han, Alex Gu, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, 2024. arXiv:2403.07974.
- [10] Haorui Li et al. When is the same model not the same service? a measurement study of hosted open-weight LLM APIs, 2026.
- [11] Guanjie Lin, Yinxin Wan, Shichao Pei, Ting Xu, Kuai Xu, and Guoliang Xue. Behavioral consistency and transparency analysis on large language model API gateways, 2026.
- [12] William Lugoloobi, Thomas Foster, William Bankes, and Chris Russell. LLMs Encode Their Failures: Predicting Success from Pre-Generation Activations, 2026. arXiv:2602.09924.
- [13] Glen Messenger. ALPS: Activation-Based Length Prediction for Intelligent LLM Inference Scheduling, 2026. Zenodo. doi:10.5281/zenodo.19078431.

- [14] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, et al. RouteLLM: Learning to Route LLMs with Preference Data, 2024. arXiv:2406.18665.
- [15] Zhihong Pan, Kai Zhang, Yuze Zhao, and Yupeng Han. Route to Reason: Adaptive Routing for LLM and Reasoning Strategy Selection, 2025. arXiv:2505.19435.
- [16] Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, et al. CARROT: A Cost Aware Rate Optimal Router, 2025. arXiv:2502.03261.
- [17] Seongho Son, Sangwoong Yoon, Jiahua Tang, Shuhan Wang, et al. SWE-Router: Routing in Multi-turn Agentic Software Engineering Tasks, 2026. arXiv:2607.00053.
- [18] Wei Song, Zhenya Huang, Cheng Cheng, Weibo Gao, et al. IRT-Router: Effective and Interpretable Multi-LLM Routing via Item Response Theory, 2025. arXiv:2506.01048.
- [19] Tanay Varshney, Annie Surla, Michelle Xu, et al. LLM Router: Rethinking Routing with Prefill Activations, 2026. arXiv:2603.20895.
- [20] Xinyuan Wang, Yanchi Liu, Wei Cheng, et al. MixLLM: Dynamic Routing in Mixed Large Language Models, 2025. arXiv:2502.18482.
- [21] Yubo Wang, Xueguang Ma, Ge Zhang, et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark, 2024. arXiv:2406.01574.
- [22] Huanyi Xie, Yubin Chen, Liangyu Wang, et al. Predicting LLM Output Length via Entropy-Guided Representations, 2026. arXiv:2602.11812.
- [23] Cheng Yan, Wuyang Zhang, Zhiyuan Ning, et al. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space, 2026. arXiv:2601.06220.